M.Eng Final Project Report

Automated 3D Plant Phenotyping Pipeline Using RealSense Depth Sensing, Machine-Learning-Based Leaf Segmentation, and Geometric Landmark Extraction

Nyra Graham
Cornell University
M.Eng Mechanical Engineering (Robotics)
Advisor: Sunghwan Jung

December 20, 2025

Contents

1 Executive Summary **4**

2 Introduction **4**

2.1 Background 4

2.2 Problem Statement 5

2.3 Project Goals 6

2.4 Scope 6

3 System Overview **6**

3.1 Hardware Setup 6

3.2 Software Ecosystem 7

3.3 Data Flow 8

4 Point-Cloud Capture Pipeline **9**

4.1 Initial Snapshot Code 9

4.2 Revised Capture Pipeline 10

5 Preprocessing and Cleaning **10**

5.1 Axis-Aligned Cropping 11

5.2 Abandoned Noise Filters and Plane Removal 11

6 Leaf Segmentation, Clustering, and Axis Estimation **13**

6.1 Motivation 13

6.2 Simple z-Band Clustering Within ML Regions 13

6.3 Leaf Axis Estimation 14

6.4 Base and Tip Identification 15

6.5 Debug Files and Overlays 16

7 Machine-Learning Leaf Segmentation **16**

7.1 Annotation Workflow in LabelMe 16

7.2 Dataset Conversion and Model Training 17

7.3 Evaluating Segmentation Performance 18

7.4 Mapping 2D Masks to 3D Point Clouds 18

8 Visualization and Debugging Tools **19**

8.1 Trajectory Plots and 3D Views 19

8.2 PNG-to-Point-Cloud Alignment 20

8.3 Automated Quality Checks 21

9 Experimental Trials and Results **21**
9.1 Trial Description 21
9.2 Segmentation Performance 22
9.3 Geometric Landmark Detection Accuracy 22
9.4 Temporal Tracking and Oscillation Patterns 23

10 Discussion **25**
10.1 Engineering Challenges 25
10.2 Limitations 25
10.3 Contributions 26

11 Future Work **26**

12 Conclusion **27**

1 Executive Summary

Plant circumnutation and leaf motion contain information about growth dynamics and environmental response, but are difficult to measure quantitatively at scale. This project develops a complete engineering pipeline that converts raw data from an Intel RealSense D435i depth camera into structured trajectories of leaf tip and base positions over time.

The system takes as input a time series of synchronized RGB images and 3D point clouds captured in a hydroponic growth chamber. It performs minimal geometric preprocessing to preserve delicate leaf structure, uses a machine-learning model trained on LabelMe polygon annotations to segment individual leaves in 2D, and then back-projects these masks into 3D to isolate per-leaf point clouds. For each leaf, principal component analysis (PCA) is used to estimate a main axis, and geometric rules locate the leaf base and tip without any additional machine learning. A set of visualization tools then converts these point tracks into 2D and 3D plots for analysis.

Key outcomes include:

- A reliable capture and file-organization pipeline for RealSense RGB+depth data, including automatic naming, cropping control, and quality checks.
- A Mask R-CNN–based leaf segmentation model trained on a small number of labeled images, integrated into a workflow that maps 2D masks to 3D point clouds and separates overlapping leaves.
- Robust geometric algorithms for per-leaf axis estimation and base/tip extraction that operate directly on sparse, noisy 3D data.
- A set of plotting and debugging scripts that visualize leaf tip trajectories in bird's-eye view, Cartesian coordinates over time, and 3D, enabling rapid diagnosis of failures and exploration of circumnutation behavior.

Beyond this specific experiment, the pipeline demonstrates a general approach for combining depth sensing, lightweight learning-based segmentation, and geometry to extract interpretable features from thin, noisy plant structures. The codebase and how-to guide are designed so future researchers can reuse and extend the system on new trials, plant varieties, and sensing setups.

2 Introduction

2.1 Background

Plants exhibit complex motion and growth behavior over time, often on slow timescales that are hard to perceive without careful measurement. Circumnutation—the oscillatory motion

of plant organs such as stems and leaves—has been studied for over a century as a window into underlying growth processes, environmental response, and even mechanosensing. Traditional measurement approaches rely on 2D imaging, hand-traced trajectories, or markers attached to the plant, which can be invasive and do not fully capture three-dimensional structure.

Recent advances in inexpensive RGB-D cameras, such as the Intel RealSense family, have made it possible to obtain dense 3D point clouds of plants in controlled environments. However, converting these raw point clouds into per-leaf trajectories remains challenging. Leaves are thin surfaces that often appear sparsely in depth data, are partially occluded, and overlap with each other. Their geometry is non-rigid and curved, and their bases and tips do not correspond to easily thresholded coordinates.

This project addresses these challenges by designing a complete engineering pipeline that starts from raw RealSense captures in a hydroponic setup and outputs time series of 3D leaf tip and base positions for a small set of "main" leaves.

2.2 Problem Statement

The raw data produced by the RealSense D435i consists of synchronized color images and 3D point clouds saved as PNG and PLY files, respectively. In this form, the data has several limitations:

- Point clouds are noisy and contain background structures, the hydroponic hardware, and the pot and tray surfaces.
- Leaves appear as thin, sometimes incomplete surfaces, with varying point density across frames due to depth noise and lighting.
- Multiple leaves can overlap in 2D projections and in 3D, making purely geometric clustering unreliable.
- Leaf bases and tips are small geometric landmarks that are not labelled in the raw data, and that are difficult to define with simple heuristics.

The goal of this project is to construct a robust, automated, end-to-end pipeline that:

1. Consistently captures and organizes RGB and depth data over multi-hour trials.
2. Segments individual leaves using a combination of machine learning and geometric constraints, even when leaves overlap.
3. Extracts the base and tip of selected "main" leaves using a stable geometric procedure.
4. Produces per-leaf trajectories over time and tools to visualize and debug them.

2.3 Project Goals

More concretely, the project aims to:

1. Develop a reliable capture pipeline for RealSense data, including cropping control via a JSON file, scheduled acquisition using Windows Task Scheduler, and sanity checks on PLY and PNG outputs.

2. Implement minimal but precise preprocessing on PLY files, using an axis-aligned bounding box to isolate the plant while preserving thin leaf geometry.

3. Train and integrate a Mask R-CNN model that uses LabelMe polygon annotations to perform 2D leaf segmentation, generating instance masks for each visible leaf.

4. Map these 2D leaf masks into 3D to generate per-leaf point clouds, then refine them with simple clustering along the camera axis and PCA-based axis estimation to identify bases and tips.

5. Build visualization tools that take the resulting trajectories and generate 3D point plots, time-series plots of x, y, and z, and bird's-eye trajectories to interpret circumnutation behavior.

2.4 Scope

The focus of this work is on the engineering aspects of the pipeline: sensor interfacing, data management, 3D geometry, and integration of machine-learning models for segmentation. Biological analysis of circumnutation patterns is outside the main scope, although the resulting plots are briefly discussed qualitatively. The machine-learning component is limited to leaf segmentation; base and tip detection are explicitly handled by deterministic geometric methods. The report emphasizes reproducibility, with a how-to guide for running the pipeline on new trials and appendices containing the core scripts.

3 System Overview

3.1 Hardware Setup

All trials were conducted in a small hydroponic growth chamber built for the plant circumnutation study. The key hardware components are:

- **Intel RealSense D435i:** mounted above the hydroponic rail to provide a top-down or slightly oblique view of the plants. The camera streams synchronized RGB and depth data at a resolution of 640×480 and a frame rate of 30 Hz. For this project, individual snapshots are captured at discrete time intervals rather than storing continuous video.

- **Hydroponic hardware:** a cylindrical PVC rail with several planting sites, each containing a small bean plant. The rail is supported by a stand inside the reflective chamber. The sensor view typically focuses on a trio of leaves from one plant that are visible and relatively separated from neighbors.
- **Lighting and environment:** reflective foil lines the chamber walls, and artificial lighting provides approximately uniform illumination. However, small lighting changes and reflections still affect depth quality and are one source of noise.



Figure 1: Hydroponic chamber and RealSense D435i mounting configuration.

3.2 Software Ecosystem

All code is organized under the project root:

`D:\Crystal\Plant Circumnutation\Hydroponics`

The main tools and libraries are:

- **Python virtual environment** at `Hydroponics\.venv`, which contains all required packages, including `numpy`, `open3d`, `matplotlib`, `torch`, and `torchvision`.

- **Capture scripts** located in the trials root:

  `D:\Crystal\Plant Circumnutation\Current_Trials_Fall_2025`

  including `RunCaptureOnce.ps1` and `RunCapture.cmd`, which wrap a Python script that talks to the RealSense and saves PNG/PLY pairs.

- **Model code** under `model\ml2\leaf_label`, which holds:
  - LabelMe-to-COCO conversion scripts,
  - Mask R-CNN training code,
  - scripts for running the model on new PNGs and overlaying predictions,
  - the 3D debug and pipeline scripts that map leaves into PLY space.

- **Visualization scripts** such as `visualize_ply.py`, `visualize_pred_points.py`, and `plot_leaf_all_views.py`, which allow quick inspection of newly captured PLY files and final tip/base trajectories.

Development is done in Visual Studio Code. A standard workflow is to open the `Hydroponics` project folder, activate the virtual environment using

```
& "D:\Crystal\Plant Circumnutation\Hydroponics\.venv\Scripts\Activate.ps1"
```

and then run all Python commands from the integrated PowerShell terminal.

3.3 Data Flow

At a high level, the data flow for a single trial is:

```
Camera → PNG + raw PLY → Cropping (capture_bounds.json) →
2D ML Leaf Segmentation (Mask R-CNN) →
2D Masks → 3D Leaf Isolation → z-Band Refinement →
PCA Axis + Base/Tip Estimation → Trajectories + Plots
```

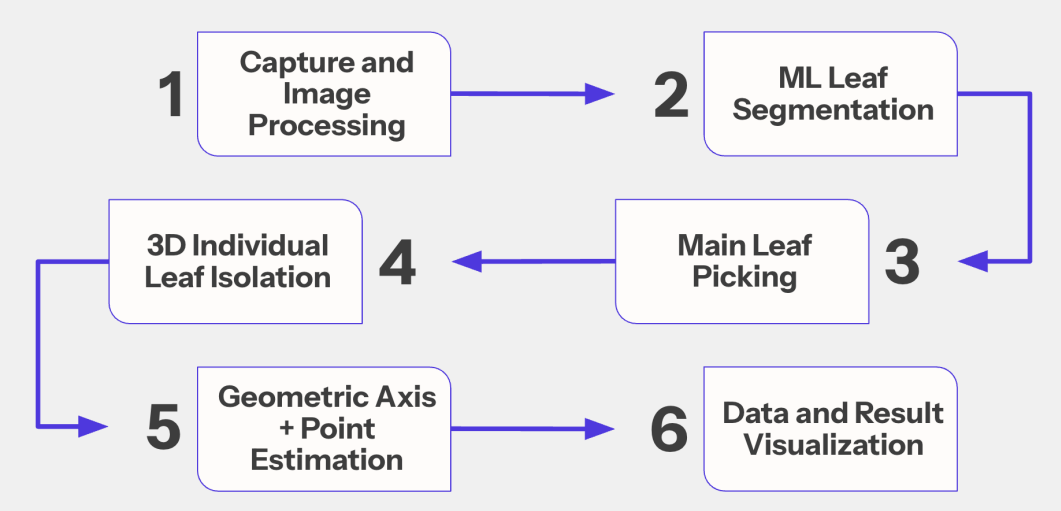


Figure 2: End-to-end pipeline from raw capture to leaf tip/base trajectories.

4 Point-Cloud Capture Pipeline

4.1 Initial Snapshot Code

The initial snapshot script interfacing with the RealSense D435i was adapted from an existing repository and configured to save a PNG and PLY file for each capture. Early in the project, several issues were encountered:

- **Depth scaling:** the reported depth scale from the RealSense API had to be inspected and applied correctly to ensure that PLY coordinates were in meters.
- **USB and reliability:** on some runs, bandwidth issues caused frames to be dropped or corrupted, resulting in empty or nearly empty PLY files.
- **OpenCV display crashes:** real-time display of RGB frames using `cv2.imshow` occasionally produced errors on the lab machine, especially when displays went to sleep.

In the earliest tests, the camera was positioned higher and aimed to capture several plants at once. This wider field of view produced strong perspective effects and overhangs: leaves closer to the camera dominated the point cloud, while more distant leaves were sparsely sampled or heavily distorted. In the resulting PLYs, bases of the target leaves were often occluded by other foliage or the rail itself. These issues motivated a change in strategy: I re-positioned the camera to focus on a single plant and a trio of main leaves, zoomed the crop window in around that region, and re-tuned the capture bounds so that those leaves filled most of the frame without being cut off.

These problems motivated a separation between a simple, robust capture pipeline and more complex visualization and processing performed offline.

4.2 Revised Capture Pipeline

The revised capture pipeline is wrapped by the PowerShell script

`RunCapture.cmd "TRIAL_PATH"`

which takes the root directory of a trial (e.g., `11_10_25 Trial`) and ensures that the following subfolders are created as needed:

- `PNG`: RGB images.
- `Point Cloud`: cropped PLY files (only the final `*_cropped.ply` is retained for each capture to save storage space).
- `NPZ`: associated metadata and intermediate arrays for each frame, including the depth image, per-pixel mask information, and camera intrinsics/extrinsics used later when mapping 2D masks into 3D point clouds.
- `Logs`: text log files for each capture run.

Each capture produces synchronized PNG and PLY files with ISO 8601 timestamps embedded in the filenames. A JSON file `capture_bounds.json` in the trials root controls the cropping applied at capture time, which ensures that only a region of interest around the plant is saved to disk.

The NPZ files are not an end product themselves, but they provide a compact way to cache all of the per-frame information that later stages of the pipeline need (camera parameters, depth arrays, and mask indices) without re-computing it from the raw RealSense output every time.

For long-term trials, capture is automated using Windows Task Scheduler. The scheduled task calls `RunCapture.cmd` every five minutes (or another frequency) for a chosen time window (typically for 9 hours each day when the lights are on), with the "Wake the computer to run this task" option enabled. This produces a time series of captures that can span many hours without manual intervention.

Before running long trials, a single test capture is performed to verify PNG framing and PLY cropping. The PNG is inspected to confirm that the target leaves are centered, and `visualize_ply.py` is used to display the corresponding point cloud. If leaves are cut off, the bounds in `capture_bounds.json` are adjusted and the test is repeated.

5 Preprocessing and Cleaning

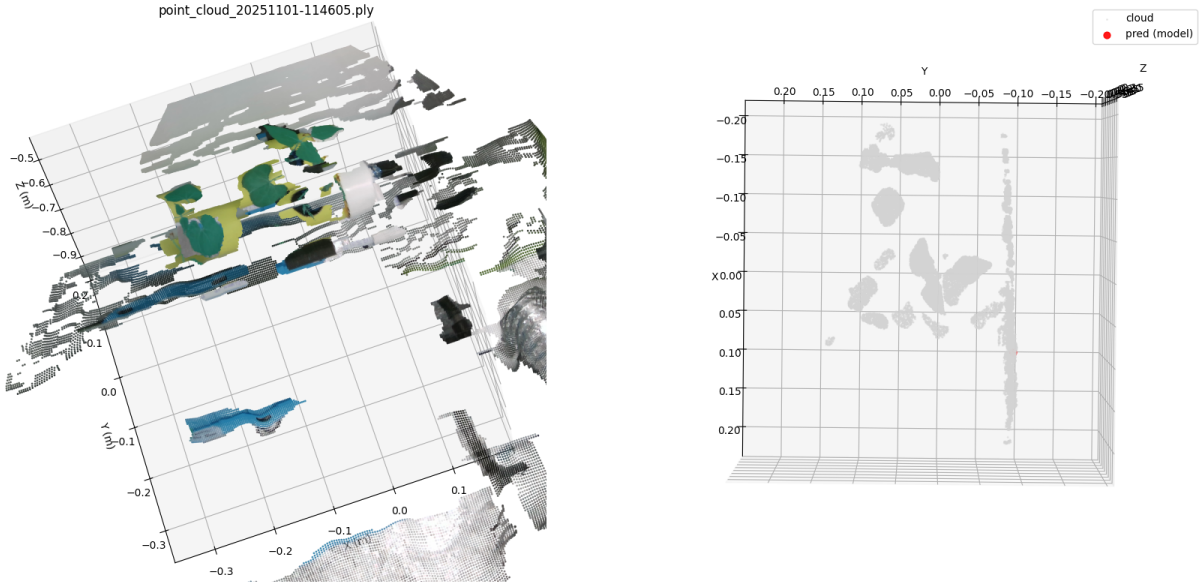
The design philosophy of the preprocessing stage is to *do as little as possible* while still isolating the plant from obvious background clutter. Early experiments with more aggressive processing—including statistical filtering, radius outlier removal, and RANSAC plane

fitting—showed that these methods can remove valid leaf points or truncate leaf bases, which is unacceptable given that leaves are extremely thin and sparsely sampled in depth.

5.1 Axis-Aligned Cropping

Cropping is performed in the camera frame using axis-aligned bounds along x, y, and z:

- Bounds are specified in `capture_bounds.json` and applied at capture time.
- The bounding box is tuned so that the target leaves and their immediate neighborhood are included, while distant background surfaces and irrelevant parts of the tray are excluded.
- If necessary, adjustments are made per trial during initial test captures.



(a) Raw PLY showing the full chamber, rail, and background clutter.

(b) Same frame after applying the axis-aligned crop around the plant.

Figure 3: Effect of axis-aligned cropping: most background points are removed while the leaves remain intact.

As shown in Figure 3, this cropping reduces point count and removes most large, distant structures, but preserves a thick margin around the leaves so that no parts of the leaf surfaces are inadvertently cut off.

5.2 Abandoned Noise Filters and Plane Removal

For completeness, several classical preprocessing techniques were tested but ultimately not adopted in the final pipeline:

- **Statistical outlier removal** and radius-based filters from Open3D were trialed to remove isolated points. While these methods do a good job at cleaning random speckles, the small, irregular neighborhoods around leaf edges often cause true leaf points to be misclassified as outliers.
- **RANSAC plane fitting** was used to attempt removal of planar backgrounds such as the table surface or back wall. However, the algorithm sometimes fit planes that intersected portions of leaves or the hydroponic rail in undesirable ways, leading to truncated geometry. Because the ML segmentation stage already handles much of the background removal, plane fitting was deemed unnecessary.

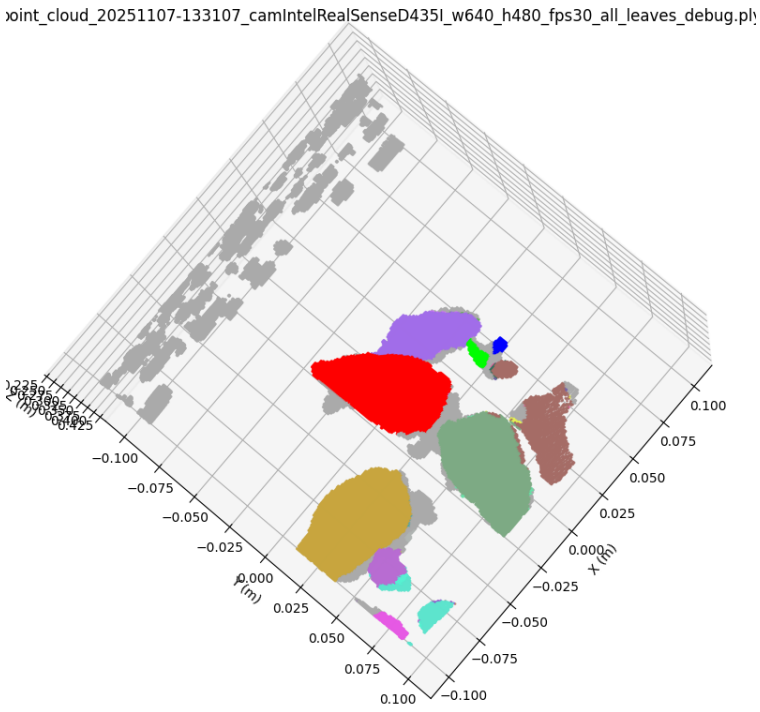


Figure 4: Failure case where the cropping window was too tight and truncated one of the main leaves. This motivated using conservative bounds with extra margin.

An example of an overly aggressive crop that removes part of a main leaf is shown in Figure 4. These experiments informed the decision to keep preprocessing intentionally minimal and to rely instead on ML masks and geometric reasoning to handle clutter.

6 Leaf Segmentation, Clustering, and Axis Estimation

6.1 Motivation

A central requirement of the project is to isolate individual leaves as separate 3D objects so that each can be tracked over time. Purely geometric clustering on the raw or cropped point cloud is unreliable in this context: overlapping leaves, variable point densities, and background structures frequently cause clusters to merge or fragment. To address this, the pipeline combines *2D machine-learning-based leaf segmentation* with *3D geometric refinement*.

6.2 Simple z-Band Clustering Within ML Regions

Once 2D leaf masks have been mapped into 3D (see Section 7), each leaf instance corresponds to a subset of 3D points. In the original design, I planned to use full 3D DBSCAN clustering to separate the main leaf surface from small fragments. In practice, external clustering dependencies were brittle on the lab machine, and a simpler approach turned out to be sufficient once leaves were already isolated by the ML masks.

The final pipeline uses a lightweight 1D clustering function `z_band_main_mask` that operates only on the z-coordinates of the points in a given leaf:

- Points are sorted by their z value (distance along the camera's viewing axis).
- The sorted sequence is split into segments wherever consecutive z values differ by more than a tolerance z_{tol}.
- The largest contiguous segment is treated as the "main" leaf band and all other points are discarded, as long as the band contains at least a minimum number of points.

In code, this corresponds to rejecting segments whose length is below a threshold `min_points` and falling back to keeping all finite points only if no sufficiently large band is found. Empirically, a tolerance of $z_{\text{tol}} \approx 0.02\,\text{m}$ and a minimum of 50 points per band worked well across the trials. Because this z-band clustering is applied *within* each ML leaf region rather than on the global cloud, it is enough to prune floating fragments above or below the true leaf surface without accidentally merging different leaves.

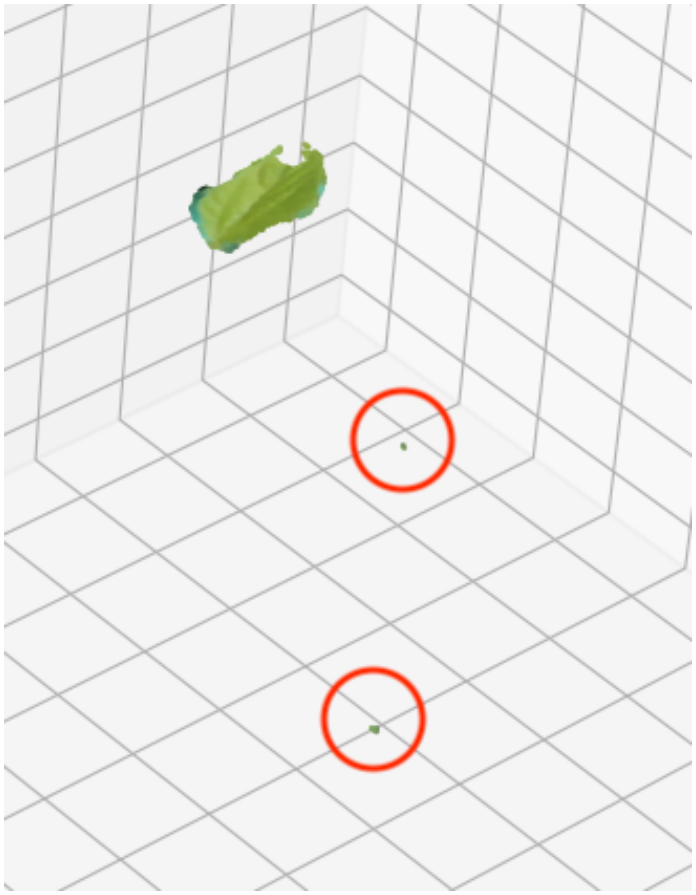


Figure 5: Example of z-band clustering on a single leaf point cloud, where the largest contiguous band along the camera's z-axis is kept as the main leaf surface.

6.3 Leaf Axis Estimation

After z-band clustering, each main leaf is represented by a set of 3D points forming a thin, elongated cloud. To compute an intrinsic coordinate system for the leaf, principal component analysis (PCA) is applied to the centered point set. The first principal component corresponds to the direction along which the points vary the most, which usually aligns with the leaf's main axis from base to tip.

Formally, given a set of N points $\{\mathbf{p}_i\}_{i=1}^{N}$, the covariance matrix

$$C = \frac{1}{N} \sum_{i=1}^{N} (\mathbf{p}_i - \bar{\mathbf{p}})(\mathbf{p}_i - \bar{\mathbf{p}})^\top$$

is computed, where $\bar{\mathbf{p}}$ is the mean point. The eigenvector associated with the largest eigenvalue of C is taken as the axis direction $\mathbf{v}$. For visualization, a line segment centered at $\bar{\mathbf{p}}$ and oriented along $\mathbf{v}$ is drawn through the leaf.

6.4 Base and Tip Identification

Once the axis vector $\mathbf{v}$ is known, each point's scalar projection along the axis is given by

$$s_i = \mathbf{v}^\top (\mathbf{p}_i - \bar{\mathbf{p}}).$$

The leaf tip and base are then defined geometrically as:

- **Tip:** the point with the maximum projection along $\mathbf{v}$ within the leaf cluster.
- **Base:** the point with the minimum projection, corresponding to the region where the leaf connects to the petiole and stem.

In practice, additional heuristics are used to ensure robustness, such as ignoring extreme outliers and requiring a minimum distance between base and tip. Importantly, this procedure replaces an earlier plan to use machine learning for base and tip detection; no separate ML model is needed at this stage.

Before settling on this geometric approach, I explored training a separate ML model to predict bases and tips directly in 3D. In practice this was not successful: each trial produced slightly different point clouds (different plants, lighting, noise patterns, and camera positions), so the model would have required a large labeled dataset covering many trials to generalize well. Labeling base and tip points in 3D is time-consuming, and the variability of the full point cloud (background, rail, and non-target leaves) makes the learning problem much harder than working with a single clean leaf. In contrast, the PCA-based axis method uses only the local geometry of one leaf cluster at a time and therefore transfers much more robustly across trials.

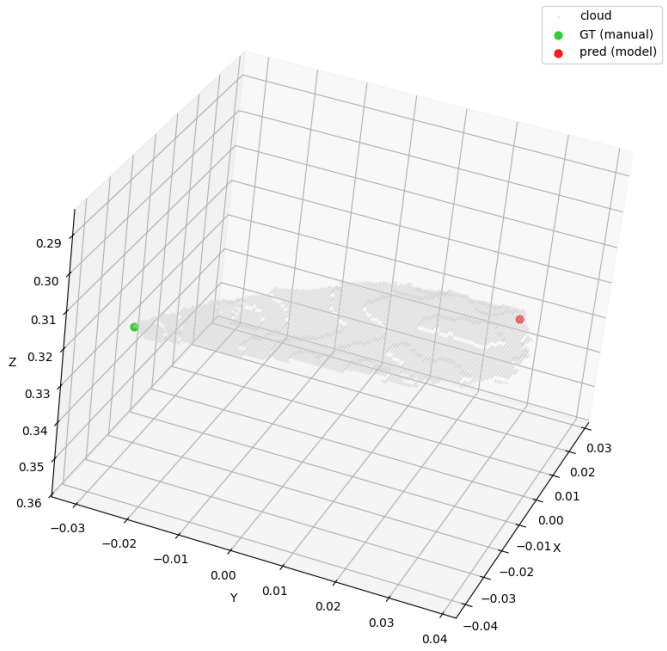


Figure 6: Geometric axis estimation and base/tip detection on a single leaf.

6.5 Debug Files and Overlays

To facilitate debugging, several scripts generate PLY files with additional information encoded as colors:

- **All-leaves debug PLYs** coloring each leaf instance with a different color, allowing visual inspection of segmentation quality.
- **Main-leaf debug PLYs** that highlight the three primary leaves selected for analysis, typically in red, green, and blue.
- **Base/tip debug PLYs** that overlay base and tip points on the full plant point cloud.

These overlays are crucial for diagnosing errors such as mislabeled leaves, incorrect main-leaf selection, or poorly estimated axes.

7 Machine-Learning Leaf Segmentation

7.1 Annotation Workflow in LabelMe

Leaf segmentation is handled by a Mask R-CNN model trained on a small, curated set of polygon annotations. The annotation process uses the LabelMe tool and follows these steps:

1. Launch LabelMe from the Python environment by running `labelme` in PowerShell.

2. Open the PNG directory for the trial of interest, which loads the first PNG and shows a list of subsequent images.

3. Use the *Create Polygons* tool to outline each visible leaf in the image. Overlapping leaves are traced individually.

4. Assign the label name `leaf` to each polygon. All leaf instances share the same class name; instance separation is handled by Mask R-CNN.

5. Save the annotations into `Hydroponics\model\ml2\leaf_label\leaves`. Typically no more than 5 PNG files per trial need to be annotated.

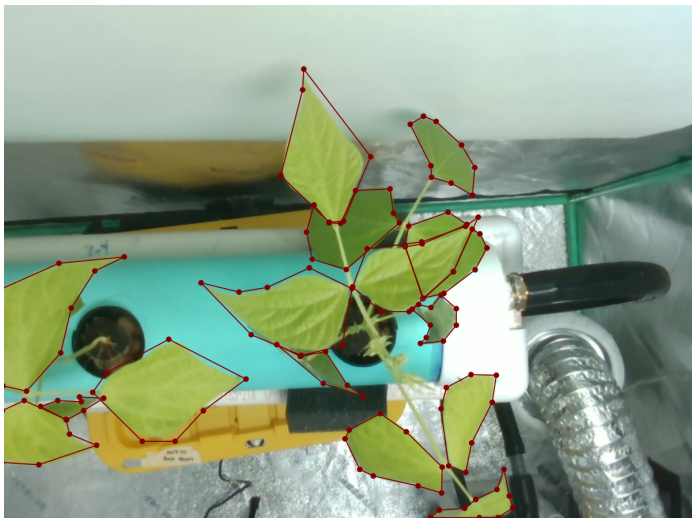


Figure 7: Example LabelMe polygon annotations for leaves.

7.2 Dataset Conversion and Model Training

The script `make_coco_from_labelme.py` converts the LabelMe JSON files into a *COCO-style* dataset. COCO is a common computer-vision format in which each image has a list of objects, and each object has a category label and a polygon or mask describing its shape. Using this format makes it straightforward to train existing instance-segmentation models.

Leaf masks are then learned with Mask R-CNN using the script `train_maskrcnn_torchvision.py`. Mask R-CNN is a neural network architecture that, given an input image, predicts both bounding boxes and pixel-accurate masks for each object instance. The training script uses the torchvision implementation with a ResNet backbone, fine-tuned on my leaf dataset. In a typical run I train for about 10 epochs with a small batch size on fewer than 20 labeled images; this keeps training time to roughly 30 minutes on the lab machine while still fitting the model to my data. Because the dataset is relatively

small, I rely on simple data augmentation (random horizontal flips, slight random crops, and mild color jitter) to expose the model to modest variations in pose and lighting and reduce overfitting.

7.3 Evaluating Segmentation Performance

Two scripts help evaluate model performance on new trials:

- `check_maskrcnn_torchvision.py` runs the model on images from a specified trial and saves overlay images to `quick_model_overlays_torchvision`. These overlays show predicted masks colored and overlaid on the RGB image, as in Figure 8.
- `debug_all_leaves_3d.py` aggregates ML predictions across PNG+PLY pairs, converts the 2D masks to 3D points, and generates debug PLYs showing all detected leaves in different colors. As part of this step, the script ranks leaf clusters by a combination of point count, proximity to the center of the crop window, and proximity to each other along the z-axis (weighting higher z ranges as more likely) and automatically selects three "main" leaves for analysis. These main leaves are highlighted in bright red, green, and blue, which makes it easy to confirm that the algorithm has picked the intended trio for that trial.

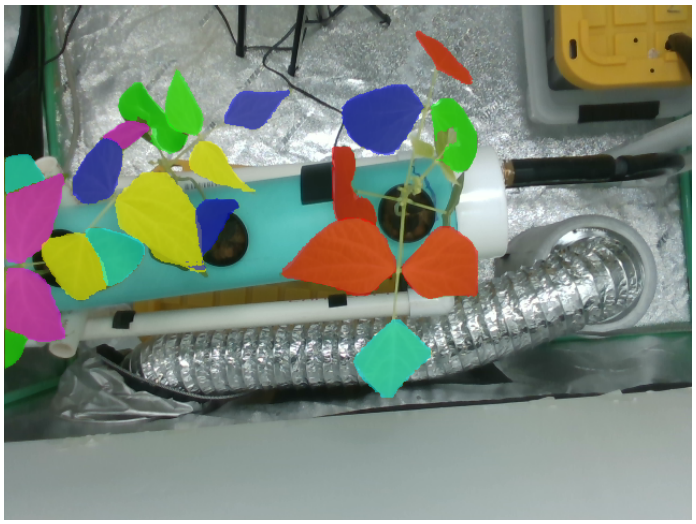


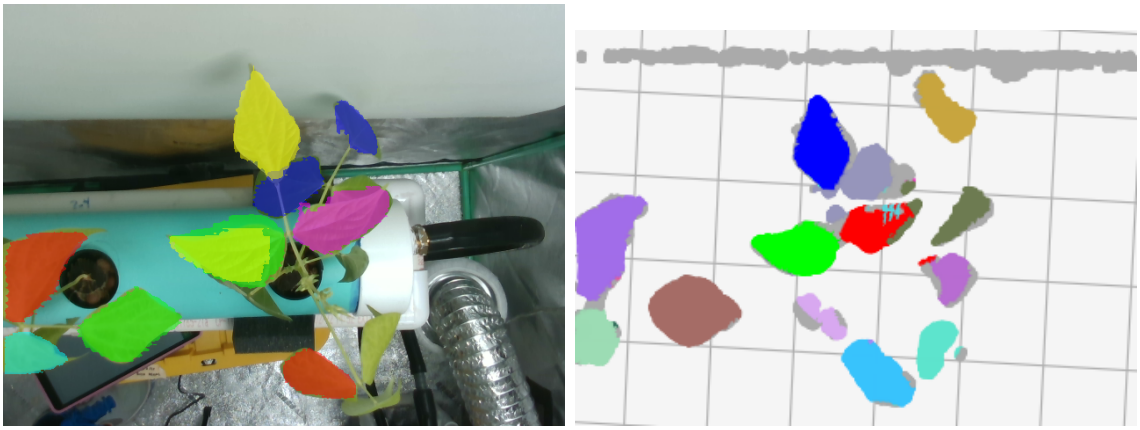
Figure 8: Mask R-CNN leaf segmentation overlaid on an RGB frame.

7.4 Mapping 2D Masks to 3D Point Clouds

For each PNG+PLY pair, the camera intrinsics and extrinsics stored in that frame's NPZ file allow correspondence between image pixels and PLY points. The pipeline uses this relationship to map 2D masks into 3D:

1. Each leaf instance mask is treated as a binary image.
2. For each 3D point in the PLY, the corresponding pixel in the PNG is found via projection.
3. Points whose pixels lie inside a given mask are assigned to that leaf instance.

This mapping yields a set of per-leaf 3D point clouds that serve as input to the clustering and axis-estimation procedures described earlier. Figure 9 shows an example of corresponding 2D masks and 3D colored clusters.



(a) 2D ML mask. (b) 3D labeled point cloud.

Figure 9: Mapping 2D ML segmentation masks to 3D point clouds.

8 Visualization and Debugging Tools

The final outputs of the pipeline are stored in a small set of human-readable files. For each trial, `run_leaf_oscillation_pipeline.py` writes a CSV under `trial_analysis` with one row per leaf per time step; the columns include the timestamp, leaf ID, and the (x, y, z) coordinates of both the base and tip. Intermediate debug outputs are saved as PLY point clouds (for visual inspection in Open3D or the custom viewers) and PNG figures (plots and overlays) under various subfolders. The scripts in this section are responsible for turning those CSV and PLY files into interpretable visualizations.

8.1 Trajectory Plots and 3D Views

The script `plot_leaf_all_views.py` is responsible for converting the CSV tracks produced by the main pipeline into informative visualizations. Given a trial ID, a CSV file path, and an optional leaf filter (1, 2, 3, or `None` for all leaves), it can generate multiple types of plots:

- A 3D plot showing the trajectories of any selected leaf tips over the chosen time window.
- Time-series plots of $x(t)$, $y(t)$, and $z(t)$ for each selected leaf.
- A bird's-eye x vs. y plot that emphasizes lateral motion for the same set of leaves.

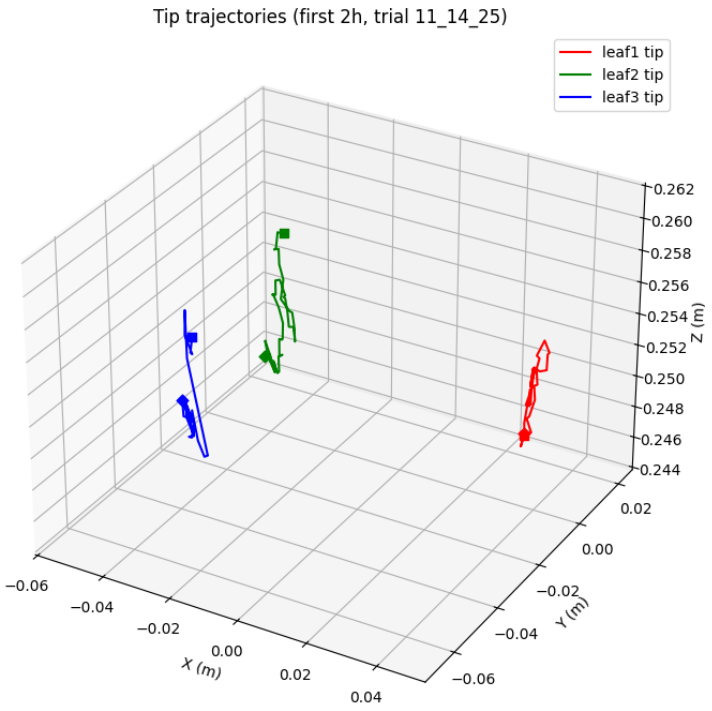
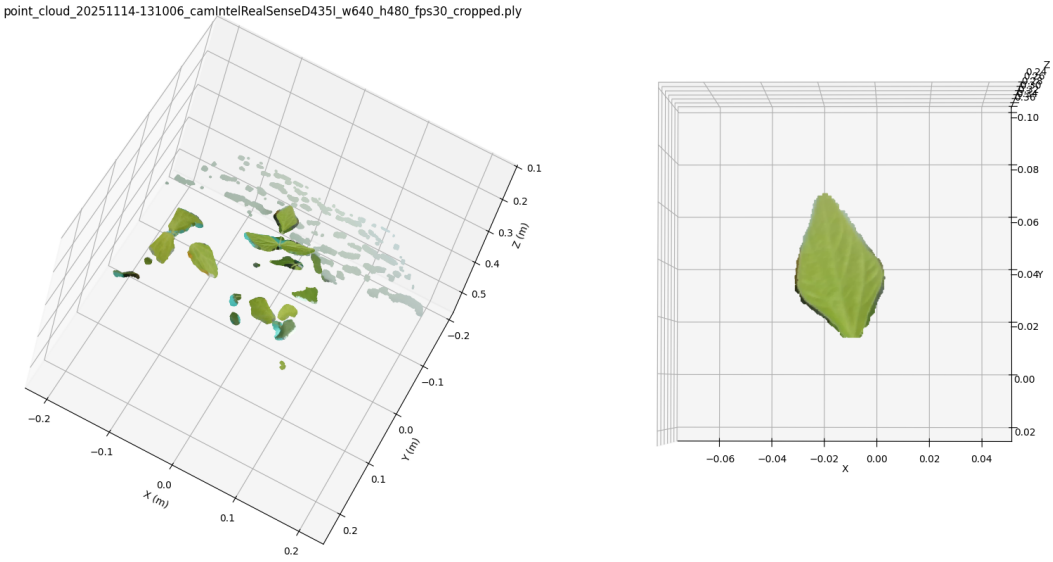


Figure 10: Example 3D trajectories for three leaf tips over the first two hours of a trial.

These plots help identify circumnutation patterns and also serve as a diagnostic tool. For example, discontinuities or sudden jumps in the trajectory may indicate tracking failures or mis-assigned leaves.

8.2 PNG-to-Point-Cloud Alignment

To verify that PNG and PLY data are properly aligned, and to diagnose cropping problems, `visualize_ply.py` can optionally show a paired view of the 3D point cloud and the corresponding color image. In the early stages of the project, this viewer was essential for tuning the capture bounds and confirming that each trial's region of interest was correctly framed.



(a) Entire PLY file. (b) Isolated leaf.

Figure 11: Debug PLYs with RGB overlay.

8.3 Automated Quality Checks

Several lightweight checks are integrated into the pipeline:

- Detection of nearly empty PLY files, which are skipped and flagged for inspection.
- Assertions on the number of main leaves detected by the ML model and 3D clustering.
- Warnings when base and tip distance falls below a threshold, which can indicate degenerate axes or mis-segmented leaves.

These checks help catch errors early and reduce the time spent manually inspecting bad frames.

9 Experimental Trials and Results

9.1 Trial Description

Multiple trials were conducted during Fall 2025 using the pipeline described above. Each trial corresponds to a specific date and plant configuration, with captures taken every five minutes over a multi-hour window. Multiple trials were conducted during Fall 2025, typically 24-36 hours long with captures every 5 minutes (roughly 250–450 frames per trial). For most trials, a trio of "main" leaves was chosen for detailed analysis based on visibility and separation from other foliage.

9.2 Segmentation Performance

Segmentation performance was assessed qualitatively using the overlay images produced by the Mask R-CNN evaluation scripts and the 3D debug PLYs. On well-lit trials where leaves were clearly visible, the model typically detected all major leaves with clean instance masks. Figure 12 shows an example of high-quality 3D segmentation.

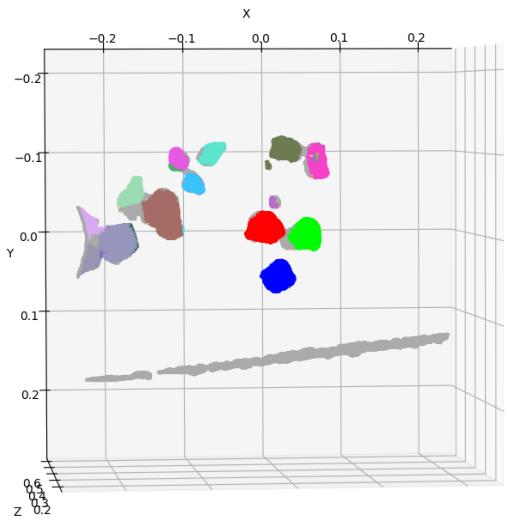


Figure 12: Example of a well-labeled 3D point cloud with distinct colors for each leaf.

In more challenging conditions—such as sparse point clouds, partially occluded leaves, or low contrast between leaves and background—the model occasionally merged nearby leaves into a single mask or missed small leaves. In these cases, retraining the model with a few annotated images from the problematic trial significantly improved performance.

9.3 Geometric Landmark Detection Accuracy

The geometric base and tip detection algorithm was evaluated by manually marking ground-truth bases and tips on selected frames and comparing them to algorithm outputs. In successful cases, the predicted base and tip were visually very close to the manual annotations, as illustrated in Figure 13. In some failure cases, extreme sparsity or unusual leaf curvature caused the PCA axis to align poorly with the actual leaf centerline, leading to base/tip errors.

Overall, for the main leaves chosen in good-quality trials, the geometric approach provided stable and interpretable base and tip positions without the need for a separate ML model.

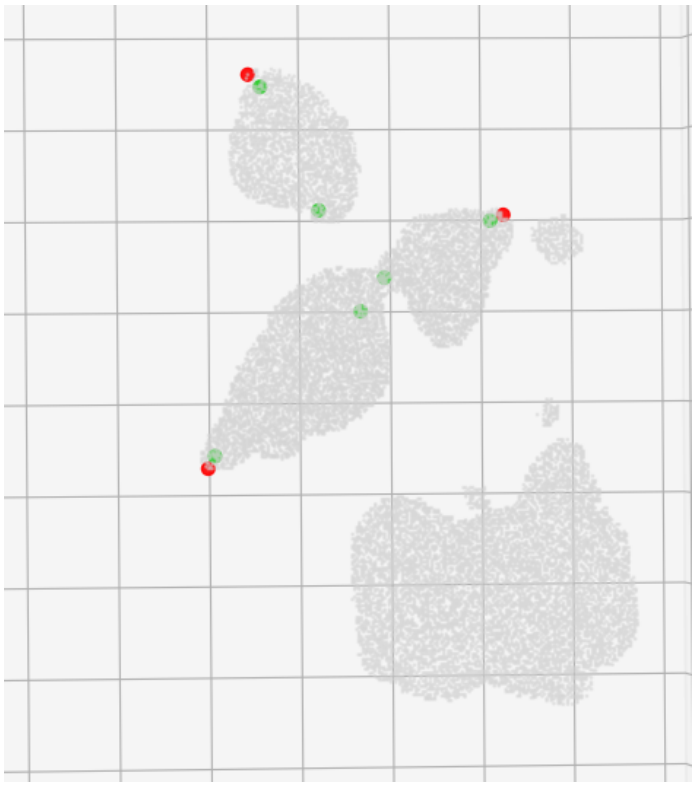
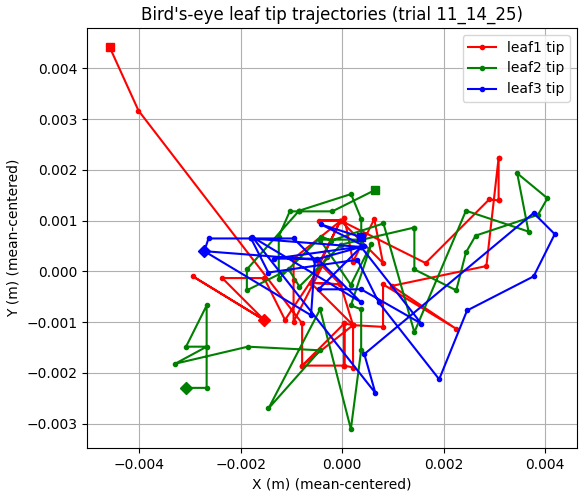


Figure 13: Example of multi-leaf tip predictions (red) compared to manual ground truth (green).

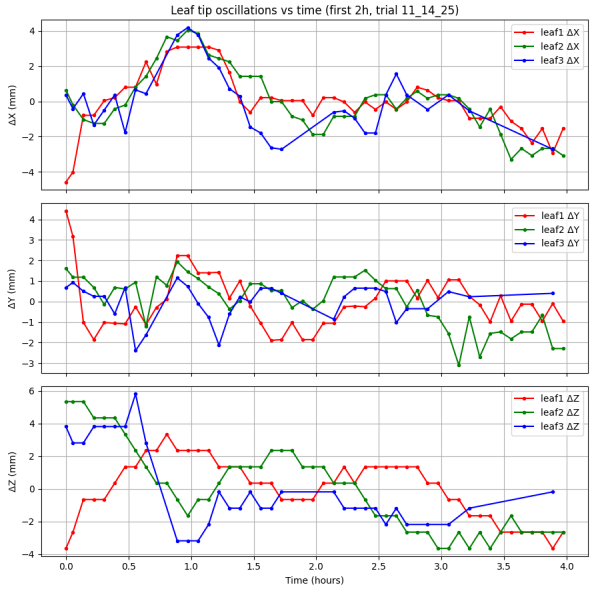
9.4 Temporal Tracking and Oscillation Patterns

The final output of the pipeline is a CSV file stored under `trial_analysis` for each analyzed trial. Each row records the time, leaf ID, and the 3D coordinates of the leaf base and tip. Using `plot_leaf_all_views.py`, the trajectories reveal a variety of behaviors:

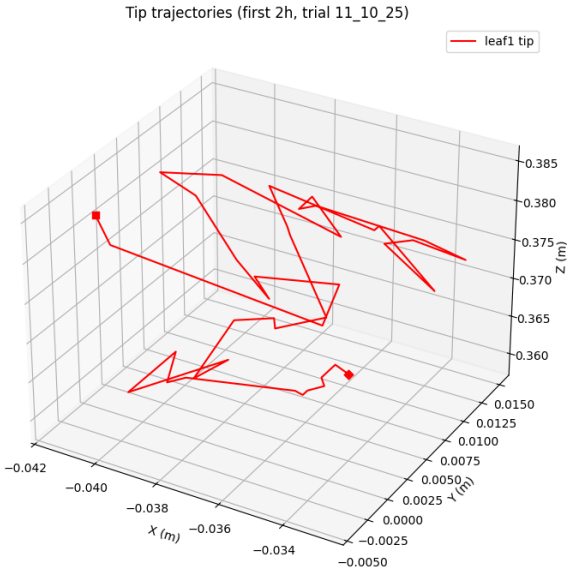
- Smooth, low-amplitude oscillations in one or two directions for healthy leaves.
- Larger, more irregular motions for leaves experiencing growth or interactions with neighboring leaves.
- Occasional discontinuities corresponding to frames where leaves moved out of view or were mis-segmented.



(a) Bird's-eye (x–y) trajectories of three leaf tips.



(b) Δx, Δy, and Δz vs. time for the same three leaves.



(c) 3D trajectory of one leaf tip over the first two hours of trial 11_10_25.



(d) Bird's-eye (x–y) trajectory of the same single leaf tip.

Figure 14: Example leaf tip oscillation trajectories. (a)–(b) show multi-leaf motion for three main leaves; (c)–(d) show the corresponding 3D and bird's-eye motion for a single main leaf.

These plots demonstrate that the pipeline can capture subtle leaf motions that would be difficult to quantify manually, providing a foundation for future biological analysis.

During analysis I observed a small number of abrupt jumps in the trajectories that did not correspond to any plausible biological motion. In the raw PNGs, these frames coincided with times when the chamber doors were opened or when the camera support was bumped slightly. Because the leaf movements are on the millimeter-scale, even small shifts in camera pose translate into noticeable discontinuities in the reconstructed (x, y, z) coordinates. A more rigid and isolated camera mount (for example, fixed to the chamber frame rather than hanging down) would help eliminate these artifacts in future experiments.

10 Discussion

10.1 Engineering Challenges

Several major challenges were encountered over the course of the project:

- **RealSense capture reliability:** early experiments suffered from depth scaling confusion, USB bandwidth issues, and occasional empty PLYs. Careful logging, simple viewing tools, and repeated test captures were essential to stabilizing the capture pipeline.
- **Machine-learning pipeline overhaul:** the initial plan to use ML for base and tip detection proved difficult to support with limited labeled data and complex geometry. Pivoting to ML for leaf segmentation and using geometry for base/tip was a significant conceptual shift requiring changes to annotations, model training, and the 3D pipeline.
- **3D segmentation instability:** early experiments with full 3D clustering (e.g., DBSCAN) alone could not reliably separate overlapping leaves in noisy point clouds. The combined ML segmentation plus simple z-band refinement was necessary to obtain robust per-leaf clusters.
- **Trial-to-trial variability:** differences in plant geometry, lighting, and noise across trials complicated the choice of fixed parameters. The pipeline had to be flexible enough to handle these variations, often with small per-trial adjustments.

10.2 Limitations

Despite its successes, the current pipeline has limitations:

- The PCA-based axis estimation assumes a roughly straight leaf centerline. Strongly curved or twisted leaves violate this assumption and can yield inaccurate base and tip estimates.

- The ML model is trained on a relatively small dataset, making it sensitive to large changes in plant species, lighting, or background appearance.
- Temporal consistency is not explicitly enforced; each frame is processed independently. Smoothing and tracking methods could reduce flicker in trajectories.
- The pipeline focuses on a small number of main leaves. Extending it to whole-plant or multi-plant analysis would require additional engineering.

10.3 Contributions

Within these constraints, the project makes several concrete contributions:

- A fully documented, end-to-end pipeline from RealSense capture to per-leaf tip and base trajectories.
- A practical method for combining Mask R-CNN leaf segmentation in 2D with 3D point-cloud processing to handle thin, overlapping plant structures.
- A robust geometric algorithm for leaf base and tip detection that does not rely on additional machine learning.
- Visualization and debugging tools that make it easy to inspect new trials, identify failure modes, and iteratively improve the system.

These tools and methods are directly reusable by other members of the research group and provide a solid foundation for future work on automated plant phenotyping.

11 Future Work

Several directions could further improve and extend this pipeline:

- **Curved-axis modeling:** replacing PCA with a method that fits a curved centerline (e.g., polynomial or spline fit) could better capture the shape of strongly curved leaves and improve base/tip estimation.
- **Temporal smoothing and tracking:** incorporating filters or tracking algorithms over time would reduce noise in trajectories and help bridge occasional segmentation failures.
- **Expanded ML dataset:** collecting and labeling more images across different plant stages and lighting conditions would improve segmentation robustness and increase generalizability to other plant types.

- **Real-time integration:** porting parts of the pipeline to ROS 2 or a similar framework could enable real-time phenotyping or closed-loop experiments, where leaf motion informs control actions or environmental changes.
- **Higher-level phenotyping metrics:** building on the trajectories, future work could compute circumnutation amplitude, frequency, and direction statistics, and relate them to environmental conditions.

12 Conclusion

This project demonstrates that it is feasible to build a robust, reusable pipeline for extracting per-leaf 3D trajectories from noisy RGB-D data using a combination of machine-learning-based leaf segmentation and geometric processing. By intentionally limiting preprocessing, using ML where it provides the most leverage (2D segmentation), and relying on interpretable geometric reasoning for base/tip detection, the pipeline balances complexity and reliability.

The resulting system captures subtle leaf motions over hours-long trials and converts them into interpretable plots that can support future biological analysis. The codebase, documentation, and how-to guide are designed so that other researchers can run the pipeline on new trials with minimal setup, extend it to new plant types, and explore additional phenotyping questions.

Appendices

Appendix A — Source Code Overview

This appendix summarizes the major scripts that make up the full phenotyping pipeline. The goal is not to provide full code listings, but to document the structure of the system and clarify the function of each component so that future users can navigate and reuse the codebase.

All files referenced below are located under the project root:

```
D:\Crystal\Plant Circumnutation\Hydroponics
```

A.1 Capture and Data Management Scripts

- `RunCaptureOnce.ps1` Wrapper script that performs a single RealSense capture. It calls the Python snapshot script, creates timestamped PNG and PLY files, applies the crop defined in `capture_bounds.json`, and writes log entries.
- `RunCapture.cmd` Main capture driver used by Windows Task Scheduler. This script ensures required trial subfolders (PNG, Point Cloud, NPZ, Logs) exist, then calls the snapshot script repeatedly. It is designed for automated multi-hour trials.

- `capture_bounds.json` User-edited configuration file defining the axis-aligned cropping region in (x, y, z) applied at capture time. Adjusted per trial to ensure leaves are fully visible and background clutter is minimized.

A.2 Machine-Learning Segmentation Scripts

- `make_coco_from_labelme.py` Converts LabelMe JSON polygon annotations into the COCO dataset format. This produces a single JSON file mapping images to leaf-instance masks, suitable for Mask R-CNN training.
- `train_maskrcnn_torchvision.py` Fine-tunes a Mask R-CNN model (ResNet backbone) on the annotated leaf dataset. Supports data augmentation (flips, slight cropping, color jitter) to compensate for the small number of labeled images. Produces a trained model checkpoint.
- `check_maskrcnn_torchvision.py` Runs the trained model on PNG images from a trial and saves overlay images. Used to confirm segmentation quality and diagnose model failures.
- `quick_model_overlays_torchvision` (folder) Stores the overlay results produced by the above script.

A.3 3D Mapping and Leaf Isolation Scripts

- `debug_all_leaves_3d.py` Maps 2D ML masks into 3D using pixel–point projection via camera intrinsics and extrinsics stored in each frame's NPZ file. Produces colored PLY files showing each detected leaf instance separately. Also ranks leaf clusters and automatically selects the "main" leaves (typically three) for analysis.
- `run_leaf_oscillation_pipeline.py` The main end-to-end processing script. It:

  1. loads PNG+PLY pairs for a trial,
  2. performs 2D→3D leaf mapping,
  3. isolates individual leaf clusters using simple z-band clustering along the camera axis,
  4. estimates PCA axes,
  5. extracts base and tip points,
  6. writes a trajectory CSV under `trial_analysis`.

  This script is used once per trial to generate all downstream results.
- `visualize_main_leaves_color.py` Visualizes the output of `debug_all_leaves_3d.py` or intermediate pipeline data by rendering each leaf cluster in a unique color, enabling rapid inspection of segmentation and cluster correctness.

A.4 Visualization and Debugging Tools

- `visualize_ply.py` Lightweight Open3D-based viewer for quick inspection of PLY point clouds. Used during capture calibration to verify crop bounds and ensure leaves are not cut off.
- `visualize_pred_points.py` Displays leaf base/tip predictions overlaid on the raw point cloud. Useful for visualizing CSV points on point clouds.
- `plot_leaf_all_views.py` Converts the trajectory CSV into informative plots, including:
  - 3D tip trajectory plots,
  - time-series plots of $\Delta x(t)$, $\Delta y(t)$, $\Delta z(t)$,
  - bird's-eye x–y trajectories.

  Supports filtering by leaf ID and is used for all final figures in the report.

A.5 NPZ Metadata Files

Each capture frame includes an NPZ file containing:

- camera intrinsics (focal lengths, principal point),
- camera extrinsics (depth-to-color transform),
- depth array used to generate the PLY,
- ML mask indices for 2D→3D mapping.

These files ensure reproducibility and prevent recomputation from raw RealSense API calls.

Appendix B — Sample PLY Files

This appendix provides representative examples of the 3D point clouds generated during the pipeline, illustrating both successful captures and common failure modes. These figures are intended to help future users recognize when camera placement, cropping bounds, or RealSense depth quality need adjustment.

B.1 High-Quality Cropped PLY Example

Figure 15 shows a typical high-quality PLY produced after tuning the camera position and the axis-aligned cropping bounds. The three main leaves appear clearly, with dense point coverage and minimal interference from background surfaces.

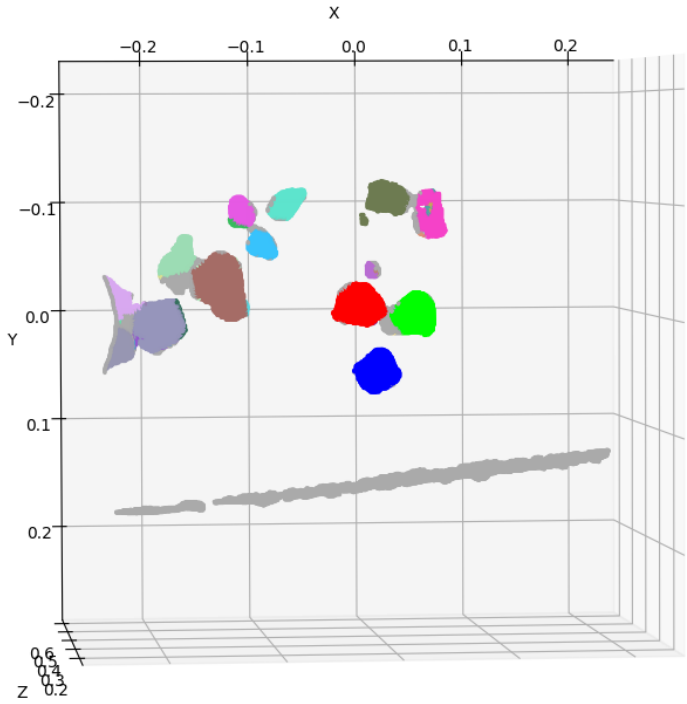


Figure 15: High-quality cropped PLY from a successful trial showing distinct leaf clusters and a clean background.

B.2 Example of Noisy or Corrupted PLY

Figure 16 shows a PLY captured during early testing when USB bandwidth issues and camera placement led to incomplete or corrupted depth frames. Such failures were characterized by:

- large floating fragments unrelated to the plant,
- streaks or vertically stretched structures,
- missing or extremely sparse leaf surfaces.

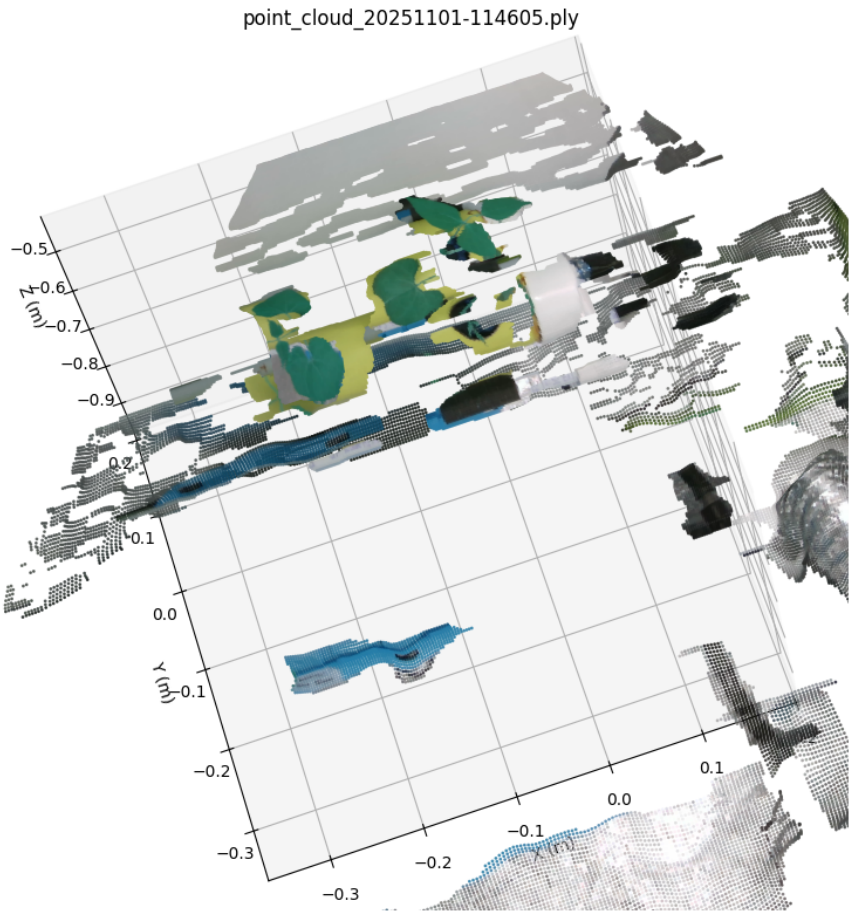


Figure 16: Example of a noisy or corrupted PLY produced during early RealSense testing.

B.3 Example of Cropping Bounds Too Small

Figure 17 demonstrates a failure mode where the user-defined cropping window was too tight, resulting in a portion of a leaf being cut off at capture time. Adjusting the `capture_bounds.json` file and re-centering the camera typically resolved this issue.

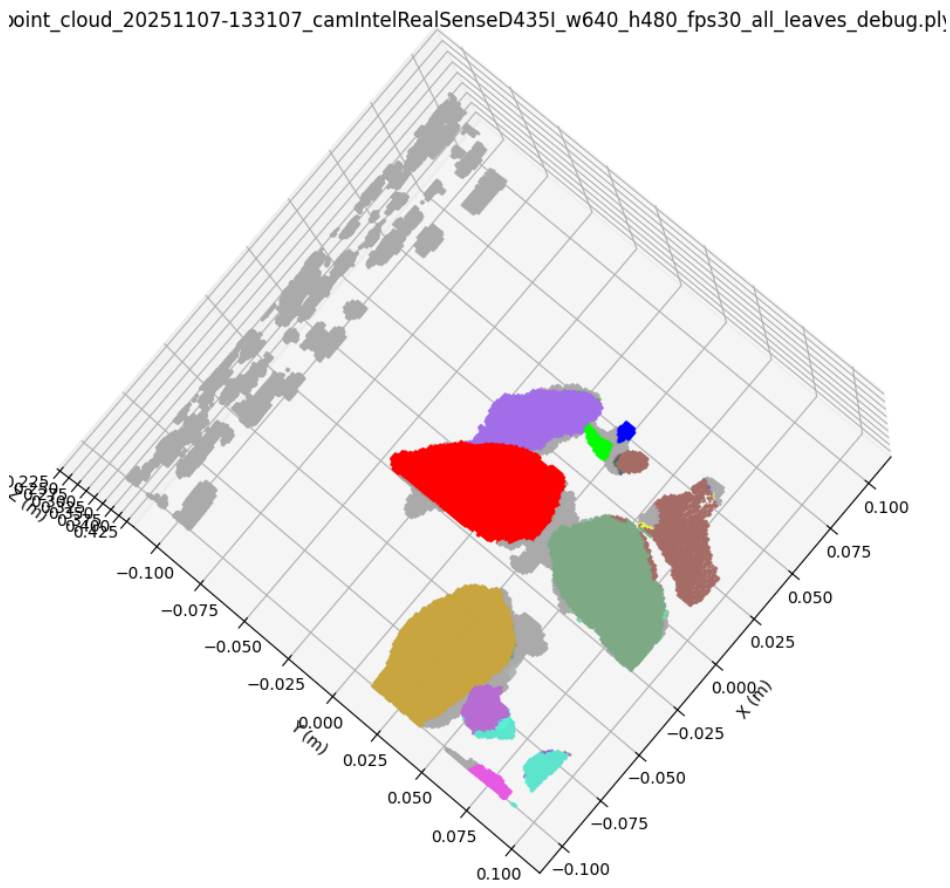


Figure 17: Example capture where the cropping bounds were too small, causing part of a leaf to be removed.

B.4 Raw PLY Before ML-Based Cleaning

Figure 18 shows a raw cropped PLY *before* ML-based masking and point-cloud isolation. The background rail, reflective foil, and chamber surfaces are still visible. This illustrates why simple global clustering was insufficient and why 2D ML segmentation was necessary for reliable leaf isolation.

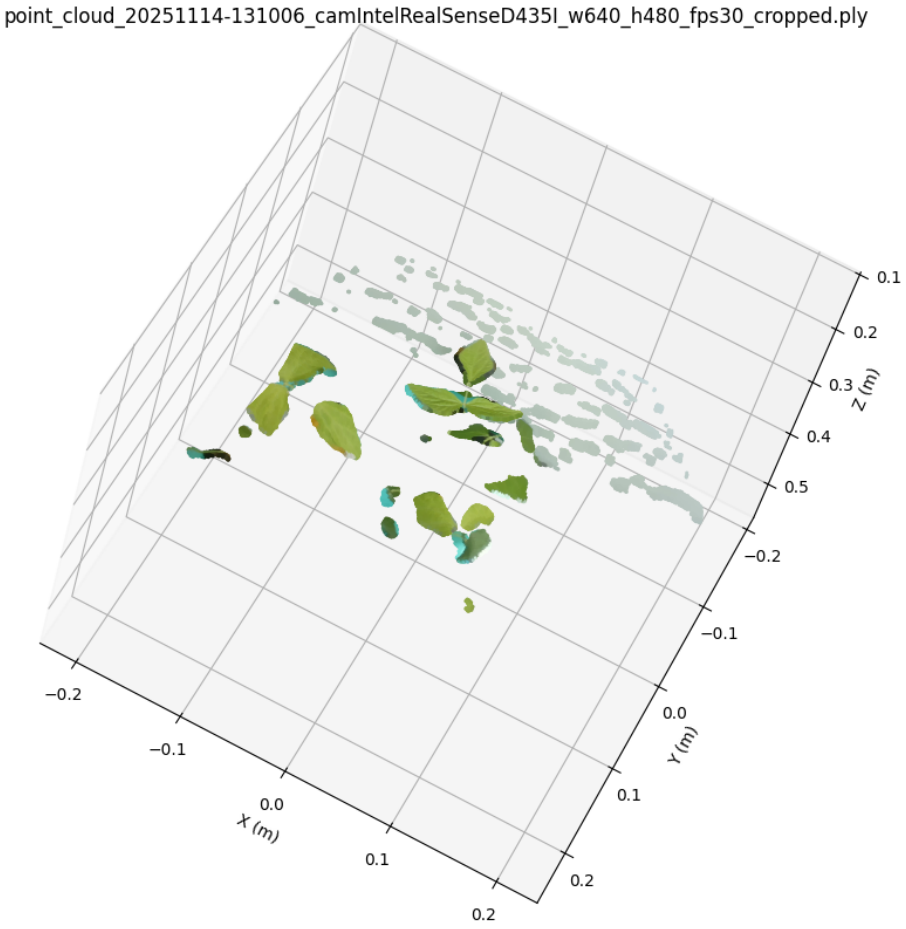


Figure 18: Raw cropped PLY showing the plant along with surrounding background surfaces prior to ML segmentation.